

2026 Korea International Friendship Competition

Cloud Computing

Test Project — Day 1

Independent Test Project Designer: Juhyeok JH Bae

July 2026

Contents

- 1. Task Description
- 2. Background
- 3. Initial State
- 4. Stages
- 5. Tasks
 - 5.1 Common Requirements
- 6. Technical Details
- 7. Service Details
 - 7.1 Database
 - 7.2 Applications
 - 7.3 Reference Documents

1. Task Description

Giftree operates a mobile gifting platform. As Giftree's cloud engineer, you are tasked with rebuilding the existing on-premises system as a cloud-native architecture on AWS.

In this test project you will build (1) a serverless ingestion API (FeedbackOps) that collects user satisfaction feedback, (2) a web service and APIs on Amazon EKS — with automatic scaling for surge traffic and cluster security policies — and (3) a CI/CD pipeline for the frontend static website plus a real-time delivery pipeline for order events.

Every component must follow the technical constraints stated in this document. Marking is performed through functional verification, load testing, and policy validation.

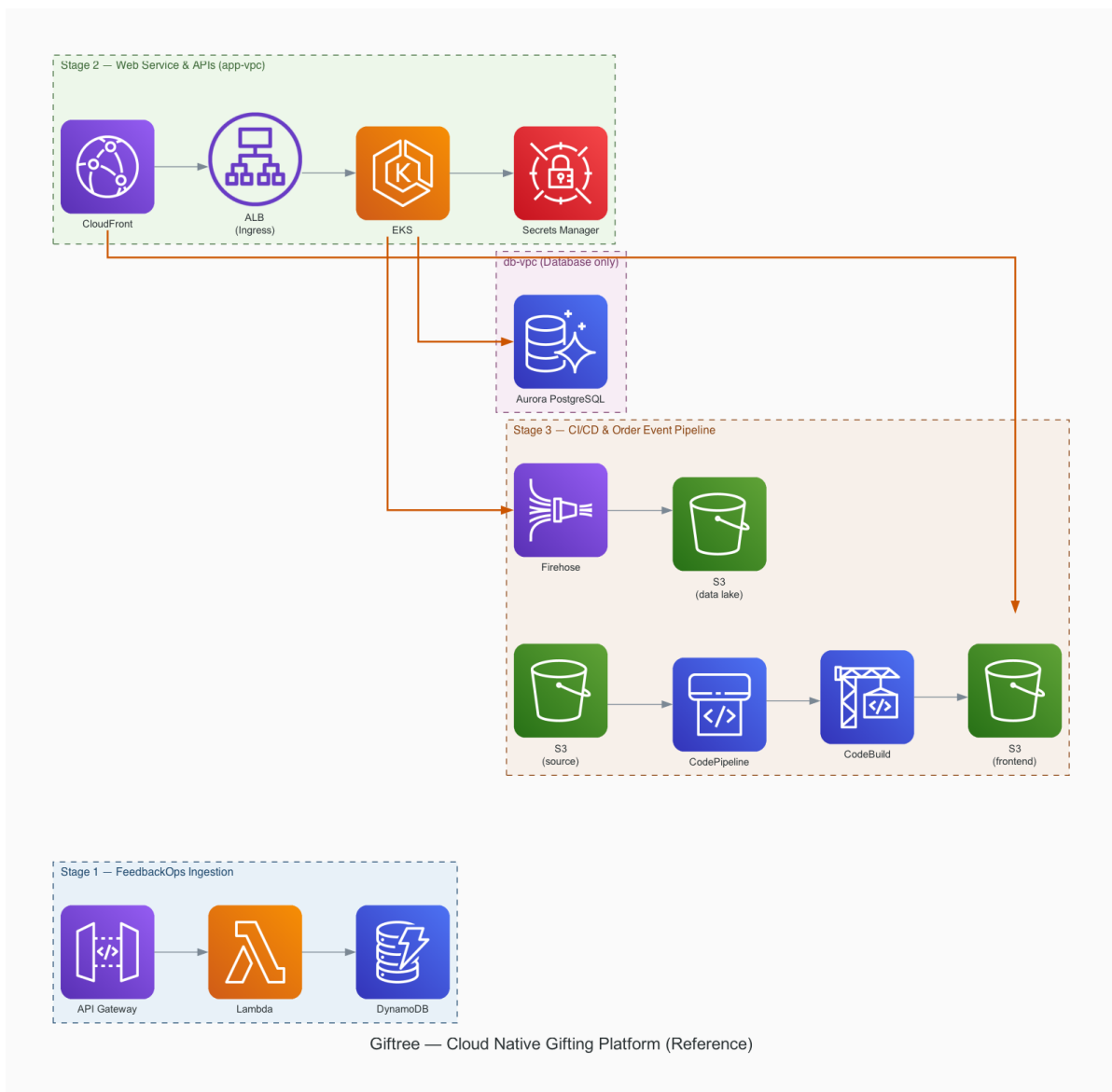
2. Background

Giftree, founded in 2019, is a mobile gifting platform that processes more than 1.2 million gift transactions per month across 300+ partner brands. Users pick a product on the web or mobile app and send it to a friend, and recipients can leave satisfaction feedback (good / bad) about the item they received.

On holidays such as Lunar New Year, Parents' Day, and Christmas, traffic surges to more than ten times the normal level. The monolithic system in the on-premises data center could not handle it and suffered a four-hour outage during the last Lunar New Year holiday. In addition, some of the satisfaction feedback left by users was lost due to overloaded receiver servers, so the data needed to improve the service could not be collected.

Management has set four strategic goals: (1) a service that scales elastically with surge traffic, (2) lossless collection of user feedback, (3) data-driven decision making through real-time delivery of order data, and (4) repeatable, automated deployments.

To achieve this, Giftree decided to move entirely to AWS. Your mission is to build the core platform within the competition time by combining container orchestration (EKS), serverless (API Gateway, Lambda), managed databases (Aurora, DynamoDB), a CDN (CloudFront), and streaming delivery (Amazon Data Firehose).



[Figure 1] Giftree system architecture overview (for reference)

CAUTION: Please use the us-east-1 region.

3. Initial State

The following are pre-provisioned. You create every other resource.

- Application artifacts: gift-shop-api, gift-gacha, frontend.zip, init.sql
- Infrastructure: VPCs, CloudFront

4. Stages

Stage 1 — FeedbackOps Feedback Ingestion (Serverless)

Gift recipients leave satisfaction feedback about items. The FeedbackOps path collects a session ID (`session_id`), an item name (`item_name`), and a verdict (`good` | `bad`). Amazon API Gateway accepts the feedback request and invokes a Lambda function, which assigns the receive time as `feedback_time` and stores the record in a DynamoDB table.

Stage 2 — Web Service and APIs (EKS)

User traffic enters through the default domain of the CloudFront distribution (`dXXXX.cloudfront.net`). CloudFront forwards static content and API requests to their respective origins. Two applications (`gift-shop-api`, `gift-gacha`) are deployed on the EKS cluster and exposed through an ALB. `gift-shop-api` is a provided application; it uses Aurora PostgreSQL and retrieves database credentials from Secrets Manager. `gift-gacha` is a provided API that returns a random gift; it must keep serving as traffic surges. Kyverno policies and VPC-CNI network policies enforce the security baseline on the cluster. `gift-shop-api` publishes an event to the Firehose stream whenever an order is created.

Stage 3 — Frontend CI/CD and Order Event Pipeline

Frontend deployment is automated with the AWS Code series. When `frontend.zip` is uploaded to the source bucket, CodePipeline starts, builds with CodeBuild, synchronizes the output to the static hosting bucket, and invalidates the CloudFront cache. Meanwhile, order events sent by `gift-shop-api` are received by an Amazon Data Firehose stream and delivered to a data lake bucket.

5. Tasks

1. Networking: place only the relational database in `db-vpc`. Place every other resource in `app-vpc`.
2. Feedback ingestion API: create a DynamoDB table. Deploy a Lambda function that stores the feedback. Wire API Gateway to invoke it. There is no authentication. The request body is JSON `{session_id, item_name, verdict}`. Return 400 if a required field is missing or verdict is not good/bad. Return 201 on a successful write.
3. EKS: create a cluster. Build `gift-shop-api` and `gift-gacha` into container images and push them to ECR. Deploy both applications on the cluster.
4. Database: create an Aurora PostgreSQL cluster. Load the schema with `init.sql`. Manage the credentials in Secrets Manager. The application must connect through the secret.
5. CDN: the frontend and the APIs must be reachable through the CloudFront default domain. Expose the applications on the cluster through an ALB.

6. Scalability: the service must keep responding as traffic surges. Resource usage must come back down once the load subsides.

7. Cluster security: apply two Kyverno policies in Enforce mode. One blocks ':latest' image tags. The other requires requests/limits on every container. Apply VPC-CNI network policies in the application namespace. Deny ingress by default and allow only the traffic needed.

8. Pipelines: deliver order events to a data lake bucket through a Firehose stream. Build the frontend CI/CD pipeline. Uploading the source alone must complete a deployment.

5.1 Common Requirements

The following apply to every task and are reflected in marking throughout.

- The system must be designed with security taken into account.
- Strong observability coverage can earn additional points.
- Implement cost-effective strategies to reduce the operating and infrastructure costs of the system.
- Design an architecture that can withstand potential chaos attacks.
- Every API must respond within 0.5 seconds. Find ways to improve the performance of the system.
- Applying caching where the system benefits from it — CloudFront, API Gateway, and so on — can help. Raising the cache hit rate is one good strategy.

6. Technical Details

1. Create all resources in the us-east-1 region. Other regions are not marked.
2. The provided applications and frontend.zip must not be modified. The only file you may change is config.ini, and you must not replace an application with your own implementation. Tampering scores zero for the related items.
3. POST /api/gacha on gift-gacha returns one random product. Analyze and monitor the application yourself to understand how it behaves.
4. Resource names and detailed placement are not prescribed. Marking assesses whether the design accounts for security and availability.
5. Once the database setup with init.sql is complete, data in DynamoDB, Aurora, and the data lake must not be manipulated directly through the console or CLI. From that point on, all data must be created through the application.
6. EKS must be version 1.31 or later, with managed node groups on amd64 nodes.

7. All EC2 instances must be t3.medium, including EKS node groups.
8. Aurora DB instances must be db.t3.medium.
9. Implement network policies with the EKS VPC-CNI feature. Third-party installations such as Calico are prohibited.
10. Kyverno policies must run in Enforce mode. Audit mode is not accepted.
11. Marking is based on the state of resources at the end of the competition. Do not modify or delete resources during marking. Each task is given partial credit.

7. Service Details

7.1 Database

The tables below are the minimum requirement. You may change the schema freely as long as the functionality works.

DynamoDB — feedback table (on-demand mode)

Attribute	Type	Key / Notes
session_id	S	Partition key
feedback_time	S	Sort key (ISO 8601, set by Lambda at receive time)
item_name	S	—
verdict	S	good bad

Aurora PostgreSQL — schema provided as init.sql

Table products

Column	Type	Constraint
product_id	SERIAL	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
brand	VARCHAR(50)	NOT NULL
category	VARCHAR(30)	NOT NULL
price	INTEGER	NOT NULL
created_at	TIMESTAMP	DEFAULT now()

Table orders

Column	Type	Constraint
order_id	UUID	PRIMARY KEY
user_id	VARCHAR(36)	NOT NULL
product_id	INTEGER	REFERENCES products(product_id)
quantity	INTEGER	NOT NULL
amount	INTEGER	NOT NULL
status	VARCHAR(16)	NOT NULL
ordered_at	TIMESTAMP	NOT NULL

7.2 Applications

- gift-shop-api — GET /healthz, GET /api/products, POST /api/orders
- gift-gacha — GET /healthz, POST /api/gacha

7.3 Reference Documents

- <https://docs.aws.amazon.com/eks/latest/userguide/cni-network-policy.html>
- <https://kyverno.io/docs/>
- <https://docs.aws.amazon.com/firehose/latest/dev/what-is-this-service.html>
- <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html>
- <https://docs.aws.amazon.com/secretsmanager/latest/userguide/intro.html>
- <https://docs.aws.amazon.com/codepipeline/latest/userguide/welcome.html>